The Cold-Chain Manifest: Edge-Aggregated Excursion Detection for Refrigerated Cargo Containers

Srinidhi Vutkoori
Student ID: X25173243
MSc in Cloud Computing
Fog and Edge Computing (H9FECC)
National College of Ireland
x25173243@student.ncirl.ie

***Abstract*—A refrigerated shipping container carries cargo whose worth depends on staying inside a narrow environmental envelope: storage temperature held well below freezing, humidity and door-open dwell bounded, and mechanical shock and in-container carbon dioxide kept low. What matters operationally is not the continuous stream of readings but the instant any one of them leaves its safe band, because a single sustained excursion can spoil a whole container. Detecting those instants by streaming every raw sample from every container to a distant region wastes bandwidth on values that individually seldom change a decision, and it couples the detection to a network round trip. This paper presents a continuous cold-chain monitor built as a two-tier fog-to-cloud pipeline. Five integrity signals across two refrigerated containers are produced by ten independent sensor processes; a fog relay on the depot's edge host folds each container-and-signal stream into fixed ten-second windows, reduces each window to summary statistics in a single pass, and screens the summary against the cargo's excursion limits locally, so only finished window summaries and the exceptions they trip cross the network. A serverless backend on Amazon SQS, AWS Lambda, and Amazon DynamoDB ingests those summaries, and a manifest board served through Amazon API Gateway from a FastAPI reader reports each container's current state and any open exception, with the static frontend delivered from Amazon S3. Every tier is justified against the alternatives it displaces, and the system was provisioned to a real Amazon Web Services account rather than to a local emulator; two faults that surface only on the managed runtime were found and fixed during that deployment. An automated suite of 76 tests passes, and verification against the live endpoint confirmed both containers streaming all five signals, the exception banner correctly flagging a breached container, and every pipeline health indicator reporting healthy against windows a few seconds old.**

***Keywords*—cold chain, refrigerated logistics, perishable cargo, fog computing, edge computing, Internet of Things, serverless computing, Amazon SQS, AWS Lambda, Amazon DynamoDB**

I. Introduction

A refrigerated container is a promise about an envelope. The pharmaceuticals, seafood, or fresh produce inside keep their value only while the storage temperature stays well below freezing, the humidity and the time the door spends open stay bounded, handling shock stays gentle, and the carbon dioxide that respiring cargo gives off stays low. The failure that destroys value is not a gradual one; it is an excursion, a spell during which one of those quantities leaves its safe band, and the cold-chain literature is consistent that time-temperature abuse of exactly this kind is the dominant cause of loss and of disputes over who is liable for it [1], [2]. Instrumenting a container with sensors is now cheap, and the Internet of Things makes it routine to read them continuously [3], but a stream of raw readings is not the same as an answer. The operator of a depot or a vessel does not want five live traces per container on five different scales; they want a current, fused verdict: is any container out of band right now, and if so, on which signal.

The distributed-systems question is where that verdict is formed. Sending every sample from every sensor to a distant cloud region to be judged there is wasteful in two directions at once. It spends bandwidth continuously on readings that, taken singly, rarely move a decision, and it defers the one moment that matters, the crossing of a limit, until after a round trip to the region and back. Fog computing was proposed precisely to close that gap, by inserting a compute tier between the sensors and the cloud, near enough to the sources to summarise, filter, and evaluate rules before anything travels [4], [5]; it is one expression of the broader movement of computation back toward the network edge [6], [7]. Behind that edge tier, the cloud supplies the elastic, pay-as-used model that removes the need to size a server for a peak that is idle most of the time [8], absorbing bursts and costing nothing while quiet.

This project builds that two-tier design for a scaled but faithful slice of a cold chain: two refrigerated containers, designated container-1 and container-2, each carrying the same five integrity signals, run as ten independent sensor processes. A fog relay on the depot's edge host receives their readings, groups each container-and-signal pair into a fixed time window, reduces the window to summary statistics, screens that summary against the cargo's excursion limits, and forwards only the finished window and any exception it raises. A serverless cloud pipeline persists each window, and a manifest board turns the persisted history into a per-container reading with an exception line and a temperature trend.

The work pursues four objectives, each shown on a running system rather than only described. The first is a sensor and fog layer that genuinely windows and screens five heterogeneous integrity signals at sampling and dispatch rates that are configurable rather than wired into the code. The second is a backend that scales through managed, elastic AWS primitives instead of a single always-on server. The third is a board that turns stored windows into an operational reading a depot supervisor could act on without inspecting raw values. The fourth, and the one that separates a working system from a merely plausible one, is deployment onto a real public-cloud account and verification against the live endpoint. Scope is held deliberately to these layers at a two-container scale rather than a whole fleet. The remainder of this paper is organised as follows. Section II presents the architecture and the reasoning behind each choice, with the alternatives weighed and the security posture. Section III covers the implementation, the automated tests, continuous integration, the real deployment, and the evidence gathered from the running system. Section IV closes with a critical assessment of the trade-offs, the limits, and where the work could go next.

II. System Architecture and Design

Fig. 1 shows the deployed system end to end: ten sensor processes and the fog relay sharing one edge host, a managed queue, two independently deployed functions, a key-value store, and a browser-facing bucket reached through a managed gateway.

A. Sensor and Fog Layer

Each of the ten sensor processes stands for one integrity signal on one container. On every sampling tick a process advances a bounded random walk: it adds a step drawn uniformly from a small symmetric range to its previous value and clamps the result into a physically plausible band for that signal, so consecutive readings stay close together like a real probe trace rather than independent noise. That continuity matters for the whole pipeline, because it is what makes a window's minimum, maximum, and average, and the board's trend line, resemble genuine container telemetry instead of static. The plausible bands differ sharply by signal, and that difference is the heterogeneity the design must carry: storage temperature sits in the low negative tens of degrees Celsius, carbon dioxide ranges across hundreds to a couple of thousand parts per million, and door-open dwell is measured in seconds that can run into the hundreds.

Two independent, separately configurable intervals drive each process. One sets how often a fresh value is sampled, defaulting to two seconds; the other sets how often the accumulated readings are dispatched to the relay, defaulting to ten seconds. Keeping the two genuinely separate, rather than tying one to the other, lets a fast-moving quantity be sampled often while still being shipped in economical batches. Each dispatch is a compact JSON object naming the signal type, the container, and the unit, and carrying the batch of timestamped values gathered since the previous dispatch. At the default rates a dispatch carries roughly five readings and amounts to a few hundred bytes on the wire, comfortably under a kilobyte. The process posts it over HTTP to the relay's ingest endpoint, so the input interface is an ordinary HTTP call the sensors make rather than a shared file or a direct write to storage. When a dispatch fails because the relay is briefly unreachable, the process puts the unsent batch back at the front of its buffer and retries on the next tick, so a transient outage delays readings rather than dropping them.

The fog relay is a Python service that runs as its own container on the edge host, distinct from the sensor processes and from the cloud functions. It does real work before anything leaves the edge, rather than acting as a pass-through. Its internal shape is a single-consumer pipeline, and that choice is deliberate. Incoming HTTP dispatches are placed on an in-process asynchronous queue and handed to one intake task that drains the queue, blocking for the first item of a burst and then greedily pulling whatever else is already waiting in one pass, and folds each batch into the open window. Because exactly one task ever mutates the window, the aggregation needs no lock: the queue itself serialises the concurrent HTTP arrivals into a single ordered stream, which is simpler to reason about and to test than a lock-guarded structure shared by many handler threads. Within the open window, each container-and-signal pair owns a small accumulator that keeps a running count, minimum, maximum, running total, and last value, so the window's statistics are maintained in one pass as readings arrive rather than by buffering every reading and reducing at the end.

A separate shipper task fires once per window period. When it fires, each non-empty accumulator is reduced to a summary carrying the count, minimum, maximum, average, and latest value, and that summary, not any raw reading, is screened against the cargo's excursion limits: an average storage temperature above $-15\,^{\circ}\mathrm{C}$ raises a cold-chain breach, average humidity above 85% a humidity breach, average door-open dwell above 300 seconds a door-open alert, average shock above 4 g an impact, and average carbon dioxide above 1000 ppm an air-quality warning. Screening the window average rather than each raw sample is what keeps a single noisy spike from raising a false alarm while a sustained excursion still trips. The limits are held in two deliberately separate places: a set of small screening functions dispatched by signal type that decide whether a summary trips, and a declarative catalogue of the same numbers that the board can read back for display, so the thresholds can be disclosed without the disclosure ever driving the evaluation.

The relay's output interface is a real managed queue rather than a direct write to storage. On each window boundary it batches the closed summaries and sends them to Amazon SQS in grouped calls, each carrying up to the service's per-batch limit of ten summaries, so the number of send operations stays proportional to the number of distinct signals closing rather than to the sampling rate. The failure behaviour on that send is itself a design choice worth stating plainly: the window's accumulators are cleared as the summaries are

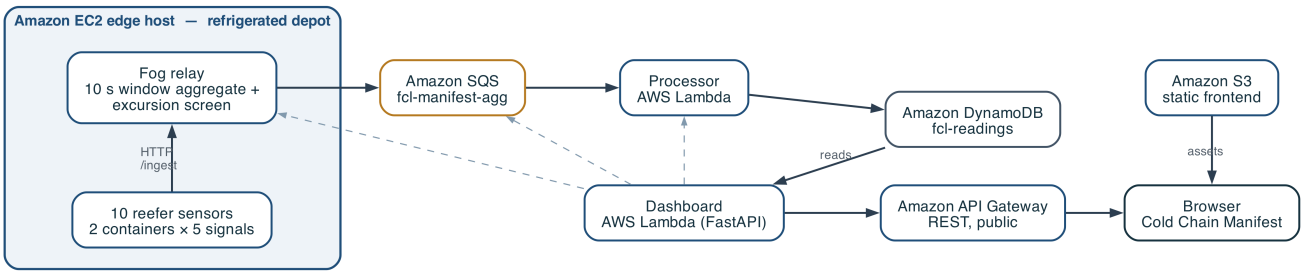

Fig. 1. Deployment topology. Ten sensor processes and the fog relay share one Amazon EC2 edge host at the refrigerated depot. Solid arrows are the ingest path, from the relay through Amazon SQS and the processor function into the Amazon DynamoDB table, and the serving path, from the dashboard function's read of the stored windows out through Amazon API Gateway to the browser, with static assets from Amazon S3. The dashed edges are the three further status sources the dashboard polls for its health strip: the relay's health endpoint, the queue's depth, and the processor function's deployment state.

drained, before the send is attempted, and the send is wrapped so that a failure is logged and the cycle continues rather than crashing the relay or blocking new readings. Because the window has already been retired, a failed send drops that one window rather than stalling the edge, a bias toward keeping the relay live and current that suits a monitoring workload where the next window is only seconds away and a momentarily missed summary is preferable to a wedged edge. A second, quieter piece of resilience sits at start-up: the relay resolves the queue's address once, retrying with exponential backoff, because on a fresh deployment the queue may not yet exist when the edge host boots.

B. Backend Layer

The cloud tier follows a single ingest path with a branch for the operator interface. A queue receives the relay's summaries, one function drains the queue and writes to storage, and a second, separately deployed function reads that storage back to answer the board. Splitting the write path from the read path is deliberate: ingestion and querying then scale and fail independently, so a surge of windows cannot slow the board and a burst of operator activity cannot back up ingestion.

Queue instead of a direct call. The relay could invoke the storage function directly and synchronously, dropping the queue and saving a hop. That option was declined. A synchronous call binds the edge to the availability and latency of the cloud function: throttle it, leave it cold, or take it down briefly, and the relay stalls or sheds data as the back-pressure propagates toward the sensors. A queue severs that coupling, absorbs bursts, retries on the consumer's behalf, and lets the relay treat a summary as delivered the moment it is enqueued [9]. On a ten-second cadence the small added latency is immaterial, while the decoupling and durability are exactly what an intermittent depot uplink needs.

Event-driven functions instead of a standing server. Both the ingestion and the interface tiers run as event-driven functions. This fits work that arrives in bursts and, for the interface, only intermittently: billing is per request rather than for idle capacity, and the platform adds and removes instances as demand shifts, with nothing reserved up front [8]. The recognised cost is the cold start after a period of idleness [10]; its effect here is small, because steady queue traffic keeps the ingestion function warm and the board's own polling keeps the interface function warm. The processor function is triggered by the queue, holds no state between invocations, and is billed only for the work it does.

One grouped send instead of one per summary. When a window closes, several container-and-signal groups usually close together, each its own summary. The relay could send them one at a time, but instead it groups every summary from a flush into batched sends. The cost that matters downstream is the number of send operations and the function invocations they trigger, not the number of readings, and a single shared window timer can close many small groups at once. Batching keeps that cost proportional to the number of distinct signals closing in a cycle rather than multiplying it out, bounded by the queue's own per-batch limit, above which the relay simply issues another batch. This is the same reduction the window aggregation performs, carried one level further, so that both the volume of data and the number of calls stay tied to how many signals exist rather than to how fast they are sampled.

Key-value store instead of a relational database. A relational engine would offer joins and open-ended queries, neither of which this workload uses. Its access pattern is narrow and known in advance: append summaries in time order, then fetch the most recent windows for a given signal. Amazon DynamoDB, an on-demand key-value store built for that mix of heavy writes and key-based reads, holds its performance as the table grows and spares the operational overhead of running a relational server, the balance the original Dynamo work set out and the managed service now carries [11], [12]. The key design follows straight from the query. Each stored window is keyed on its signal type as the partition and on a sort value that leads with the window's closing timestamp and then the container identifier, so windows for one signal are already ordered chronologically inside a single partition and the newest are read with one bounded, descending, single-partition query rather than a scan. Because the container is only the tail of the sort value and not part of the partition key, a request for one container's history over-fetches a wider page of the signal's windows and filters to that container in the reader, an explicit trade of a little read amplification for a key that keeps every

signal's windows contiguous and in order.

C. Serving Layer and Security Posture

The board reaches the read function through a managed gateway that publishes a public REST endpoint. Object storage holds the static frontend, one page, a stylesheet, the page script, and a charting library kept in the repository rather than pulled from a content-delivery network. The read function is a FastAPI application, and it is driven inside the function by a small hand-written adapter that turns each gateway event into one ASGI request and the application's response back into a gateway result, so the same application code runs unchanged whether it is served locally or invoked as a function. It exposes a small, read-only surface: a manifest endpoint giving the newest window for every signal on both containers, a readings endpoint returning a recent series for one signal to feed the trend charts, a thresholds endpoint that forwards the relay's published limits, a backend-statistics endpoint, and a health endpoint. The health response is assembled from four live sources at once, the store for the freshest window, the queue for its reachability, the processor function for its deployment state, and the relay for its own reachability, the read dependencies drawn in Fig. 1, and it returns those flags together with a pipeline flag that is true only when a window actually reached the store within the last thirty seconds, so a reachable-but-stale pipeline is distinguished from a healthy one.

On the page, the two containers are a single manifest table of one row each, showing every signal's current value with its unit, a status of either healthy or a count of open exceptions, and how long ago the row was refreshed; a header reports how many containers are tracked and how many exceptions are open, and an exception banner names any firing condition by container. Below the table, a storage-temperature trend is drawn per container so the quantity most likely to spoil cargo is the one shown over time, and a footer strip reports the four pipeline indicators and the archived record count. The exception count in the header is derived on the client from the same window data the table shows, so the banner and the table can never disagree. A fixed browser-side timer re-fetches the manifest, health, and backend statistics every 2.5 seconds and repaints from whatever the responses hold, so the view stays current without a manual reload and without any server-push channel. The page learns which gateway origin to call from a single placeholder written into it, substituted with the real endpoint when the assets are uploaded and read once by the script; run locally, where one process serves both the page and the interface, the placeholder resolves to empty and requests fall back to the same origin. The security posture is modest and deliberate for a scaled demonstration: the relay's health and threshold endpoints answer without a credential check because they expose only aggregate window state and configured limits, never a live per-sensor reading, so what they disclose is bounded to what the board already renders, while the cloud tier relies on the managed services' own access control and on scoping each function to the queue, table, and gateway it needs.

III. Implementation

The sensor processes, the fog relay, and both cloud functions are written in Python and run on the Python 3.12 runtime. The relay and the dashboard reader are FastAPI applications; the relay serves its ingest, health, and threshold endpoints under an event loop that also runs the intake and shipper tasks, while the sensor processes stay dependency-light and post their batches with the standard library's HTTP client. The AWS SDK for Python carries the queue, table, and function calls, and the frontend renders its trends with a locally vendored copy of a charting library so the page has no external dependency at load time. The edge tier is orchestrated with Docker Compose, the cloud tier is provisioned with Terraform, and continuous integration runs on GitHub Actions.

A. Testing and Continuous Integration

The system carries an automated suite of 76 tests spread across every component: the window aggregation and its arithmetic, the excursion screeners at and around each limit, the sensor's sampling and dispatch behaviour, the relay's ingest endpoint and its status routes, the batched publisher, the processor's normalisation of a queued summary into a stored item, and the dashboard reader's query shaping, health assembly, and routes. They run without any cloud connection by exercising the units against hand-written fake AWS clients, so each test can assert on the exact shape of the request a given call would issue rather than depending on a live service. The screening tests are pointed deliberately at the boundaries, checking a value one step inside a limit and one step outside it, so a threshold that should fire and one that should not are both pinned down; the publisher tests check that a flush is split into batches no larger than the queue's per-call limit; and the processor tests check the composite sort value that orders windows within a partition. The reader tests cover the manifest assembly from stored windows, the recent-series query that feeds the trend charts, the four health checks, and the request routing the deployed function performs, since that routing is code the local server never runs and so is the part most exposed to a deployment-only mistake. The suite runs on every push through GitHub Actions, so a change that breaks a screening boundary, a query shape, or the routing fails in continuous integration rather than after deployment.

Testing against fakes rather than a live queue was a deliberate choice with a cost worth acknowledging: a fake asserts on the request a call would make, not on how the managed service behaves under load, so the batched-send limit and the queue's own retry semantics were reasoned about from the service documentation [13] and then confirmed against the live deployment rather than in unit tests. Standing up a local emulator for every test was rejected as slower and still not the real service, and the gap it leaves is exactly what the live verification below is for.

B. Deployment and Live Verification

The cloud tier was provisioned to a real AWS account in a single Terraform apply that created twenty-four resources, with the account confirmed before anything was applied and the working state isolated so the apply could add only its own resources and never disturb anything already present. Describing the whole cloud tier as code, rather than clicking it together once, means the same definition builds the queue, both functions with their triggers and permissions, the table, the gateway with its single catch-all route, and the two buckets in one reproducible step, and tears them down as cleanly. The edge host runs the fog relay and the ten sensor containers under Docker Compose; the managed queue, the two functions [14], the table, the managed gateway [15], and the frontend bucket make up the cloud side. All the deployed resources carry a common prefix so the deployment is easy to identify and to remove as a unit, and a fixed public address in front of the edge host lets the read function reach the relay's health and threshold endpoints at the same location across restarts.

Two faults surfaced only once the reader ran as a real function, and both are worth recording because neither can appear in a local run. The reader's web framework depends on a component with a compiled native part, and the deployment package first shipped the copy built for the development machine; the managed runtime, on different hardware, could not import it, and the function failed at load. Rebuilding the package against the runtime's own platform fixed it. The second fault was subtler: the application mounts a static-asset directory at import time, and the deployment package did not include that directory because the frontend is served from object storage, not from the function, so the import raised before any request was handled. Making the mount tolerant of an absent directory resolved it without shipping assets the function never serves. Both are exactly the class of failure a fixture cannot reach, since a fixture imports the code on the same machine that wrote it.

Verification was carried out against the live deployment rather than a local emulator, and Fig. 2 is the deployed board during that check. Within about a minute of start-up the pipeline reported healthy end to end: the relay, the queue, the processor function, and the freshest-window recency all read healthy, and the newest window was a few seconds old on each poll. Both containers rendered all five signals with their current value; the header reported two containers tracked and, with container-2's storage temperature and door-open dwell above their limits, two open exceptions; and the exception banner named both firing conditions on the correct container. The storage-temperature trends filled and advanced per container, and the backend-statistics endpoint showed the archived record count climbing across successive polls rather than a static number, which confirms the ingestion function was draining the queue and writing on an ongoing basis. Reaching the board from a browser also exercised the cross-origin path from the storage bucket to the gateway, which returned data cleanly.

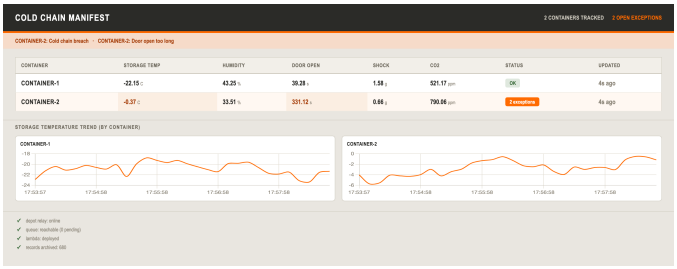

Fig. 2. The deployed manifest board reading from the live endpoint. Each container shows its five integrity signals with current value and status; the header reports containers tracked and open exceptions; the banner names the firing conditions on the breached container; the per-container charts show the storage-temperature trend; and the footer strip shows the four pipeline indicators and the archived record count.

IV. Conclusion

This project set out to build and, above all, to deploy a fog-to-cloud pipeline that turns five integrity signals across two refrigerated containers into a single live reading of whether the cargo is in band, and it met that goal on a running system rather than on paper. The edge tier does genuine work before anything travels: it windows and reduces each container-and-signal stream and screens the summary against the cargo's excursion limits locally, so only compact summaries cross the network and a breach is flagged in the window it occurs. The cloud tier scales through decoupling and elasticity rather than a pre-sized server, using a queue to absorb bursts and separate the write and read paths, event-driven functions billed only for the work they do, and an on-demand key-value store whose key design turns the board's one query into a single-partition read. The whole system was provisioned to a real account by one infrastructure-as-code apply and verified end to end against the live endpoint, with 76 automated tests guarding its behaviour and continuous integration running them on every change.

Two reflections stand out from building it. The first is that the value of the fog tier is easiest to see precisely because the signals are heterogeneous and the thing worth catching is rare: reducing five streams on five different scales to uniform per-window summaries at the edge, and screening the average rather than the raw sample, is what lets one small, uniform record type flow through the queue, the function, and the table while a real excursion still trips in the window it happens, without a spike raising a false alarm. The second is that deploying to a real account is a different claim from passing tests. The two faults that mattered here, a native dependency built for the wrong platform and a static mount that fails at import, both passed every local test and both broke the function the moment it ran on the managed runtime, because a test imports the code on the machine that built it and the platform does not. Each had to be reasoned about against production behaviour rather than assumed away, and finding them was the clearest argument in the project for verifying against the real endpoint.

The work has clear limits. It runs at a two-container scale

with simulated sensors, its excursion limits are fixed rather than learned or adjustable at runtime, and its security posture is intentionally modest for a demonstration. Natural next steps are to widen it to a fleet of containers and real sensor feeds, to move the fixed limits to a managed configuration an operator could adjust without a redeploy, to add authentication and a custom domain in front of the public endpoint, and to retain longer history for trend analysis beyond the live view. None of these changes the shape of the pipeline, which is the point: the edge-aggregated, excursion-screening design carries from two containers to many without rethinking the tiers.

REFERENCES

[1] N. Ndraha, H.-I. Hsiao, J. Vlajic, M.-F. Yang, and H.-T. V. Lin, "Time-temperature Abuse in the Food Cold Chain: Review of Issues, Challenges, and Recommendations," *Food Control*, vol. 89, pp. 12–21, 2018.

[2] M. M. Aung and Y. S. Chang, "Temperature Management for the Quality Assurance of a Perishable Food Supply Chain," *Food Control*, vol. 40, pp. 198–207, 2014.

[3] L. Atzori, A. Iera, and G. Morabito, "The Internet of Things: A Survey," *Computer Networks*, vol. 54, no. 15, pp. 2787–2805, 2010.

[4] F. Bonomi, R. Milito, J. Zhu, and S. Addepalli, "Fog Computing and Its Role in the Internet of Things," in *Proc. MCC Workshop on Mobile Cloud Computing*, 2012, pp. 13–16.

[5] A. Yousefpour et al., "All One Needs to Know about Fog Computing and Related Edge Computing Paradigms: A Complete Survey," *Journal of Systems Architecture*, vol. 98, pp. 289–330, 2019.

[6] W. Shi, J. Cao, Q. Zhang, Y. Li, and L. Xu, "Edge Computing: Vision and Challenges," *IEEE Internet of Things Journal*, vol. 3, no. 5, pp. 637–646, 2016.

[7] M. Satyanarayanan, "The Emergence of Edge Computing," *Computer*, vol. 50, no. 1, pp. 30–39, 2017.

[8] M. Armbrust et al., "A View of Cloud Computing," *Communications of the ACM*, vol. 53, no. 4, pp. 50–58, 2010.

[9] G. Hohpe and B. Woolf, *Enterprise Integration Patterns: Designing, Building, and Deploying Messaging Solutions*. Addison-Wesley, 2003.

[10] L. Wang, M. Li, Y. Zhang, T. Ristenpart, and M. Swift, "Peeking Behind the Curtains of Serverless Platforms," in *USENIX Annual Technical Conference*, 2018, pp. 133–146.

[11] G. DeCandia et al., "Dynamo: Amazon's Highly Available Key-value Store," in *Proc. ACM SIGOPS Symp. Operating Systems Principles*, 2007, pp. 205–220.

[12] M. Elhemali et al., "Amazon DynamoDB: A Scalable, Predictably Performant, and Fully Managed NoSQL Database Service," in *USENIX Annual Technical Conference*, 2022, pp. 1037–1048.

[13] Amazon Web Services, "Amazon Simple Queue Service Developer Guide," 2024. [Online]. Available: https://docs.aws.amazon.com/sqs/

[14] Amazon Web Services, "AWS Lambda Developer Guide," 2024. [Online]. Available: https://docs.aws.amazon.com/lambda/

[15] Amazon Web Services, "Amazon API Gateway Developer Guide," 2024. [Online]. Available: https://docs.aws.amazon.com/apigateway/

[16] IEEE, "Manuscript Templates for Conference Proceedings," 2024. [Online]. Available: https://www.ieee.org/conferences/publishing/templates.html

The source code and deployment configuration are available at [GitHub repository URL].